

Backend Developer — Technical Assessment

Meeting Room Booking Service

Time: 3-4 hours

Objective

Build a backend service that allows an administrator to book meeting rooms for customers.

Customers are managed in an external CRM, while room bookings are managed in your application. Every successful booking must also be created in Google Calendar.

We are evaluating your ability to work with REST APIs, third-party integrations, database design, and backend architecture.

Technology

- **Language:** Python (FastAPI / Django REST) or Node.js (Express / NestJS) — your choice
- **Database:** PostgreSQL
- **CRM — Zoho CRM** — zoho.com/crm/developer
- **Calendar:** Google Calendar API — a **single Google account/calendar** for all bookings. A service account is the simplest route (enable the Calendar API, share your calendar with the service account email). OAuth is fine too.

Set up both accounts and confirm one authenticated API call **before** you start coding .

Requirements

Setup — a seed script creating 5 meeting rooms (name, capacity, location). Create 5 test contacts in your chosen CRM; these are your customers.

1. Customers — `GET /customers` returns contacts from the CRM (name, email, company) with search by name or email. Handle the CRM's **pagination** properly. **Cache** the list rather than calling the CRM on every request.

2. Rooms & availability — `GET /rooms`; `GET /rooms/:id/availability?date=YYYY-MM-DD` returning free and busy slots for that day.

3. Create booking — `POST /bookings` (customer ID, room ID, start, end, purpose). Validate:

- **No overlap** for that room — and this must hold **when two requests arrive at the same instant** (DB constraint, lock, or transaction; explain your choice)
- Customer ID exists in the CRM

4. Google Calendar — each confirmed booking creates an event on the shared calendar (room name, time, customer).

5. List bookings — `GET /bookings`, filterable by date range, room, and customer; each result enriched with the customer's CRM details.

Authentication is not required — leave endpoints open

Failure handling (mandatory)

This is the core of the assessment. If you run short on time, cut scope elsewhere. We will test these live during the review call.

1. **Google Calendar unreachable at booking time** → the booking still succeeds; the calendar event is queued and created on recovery. A booking is never lost because a third-party API is down.
2. **The same booking request submitted twice** (double-click or retry) → exactly one booking, one calendar event.
3. **The CRM rate-limits you (HTTP 429)** → back off and retry; the customer list keeps serving from cache.

Optional bonus: a minimal frontend demonstrating the flow end-to-end — search a customer, pick a room and slot, create a booking, see it listed. Plain HTML/JS is fine; no styling expected.